
A Scalable Deterministic Global Optimization Algorithm for Training Optimal Decision Tree

Kaixun Hua

University of British Columbia
kaixun.hua@ubc.ca

Jiayang Ren

University of British Columbia
rjy12307@mail.ubc.ca

Yankai Cao*

University of British Columbia
yankai.cao@ubc.ca

Abstract

The training of optimal decision tree via mixed-integer programming (MIP) has attracted much attention in recent literature. However, for large datasets, state-of-the-art approaches struggle to solve the optimal decision tree training problems to a provable global optimal solution within a reasonable time. In this paper, we reformulate the optimal decision tree training problem as a two-stage optimization problem and propose a tailored reduced-space branch and bound algorithm to train optimal decision tree for the classification tasks with continuous features. We present several structure-exploiting lower and upper bounding methods. The computation of bounds can be decomposed into the solution of many small-scale subproblems and can be naturally parallelized. With these bounding methods, we prove that our algorithm can converge by branching only on variables representing the optimal decision tree structure, which is invariant to the size of datasets. Moreover, we propose a novel sample reduction method that can predetermine the cost of part of samples at each BB node. Combining the sample reduction method with the parallelized bounding strategies, our algorithm can be extremely scalable. Our algorithm can find global optimal solutions on dataset with over 245,000 samples (1000 cores, less than 1% optimality gap, within 2 hours). We test 21 real-world datasets from UCI Repository. The results reveal that for datasets with over 7,000 samples, our algorithm can, on average, improve the training accuracy by 3.6% and testing accuracy by 2.8%, compared to the current state-of-the-art.

1 Introduction

Decision tree, as a typical supervised learning algorithm with strong interpretability, has been applied in a wide range of fields. As a rule-based inductive model, a decision tree concludes a series of decision rules to navigate tasks such as classification or regression. The target of the decision tree is to achieve a low prediction error while maintaining the low complexity of the decision tree structure. Therefore, modelling a decision tree can be treated as an optimization problem that minimizes the training error and the tree complexity. However, even constructing an optimal decision tree with binary class is NP-complete [Laurent and Rivest, 1976]. Since the first establishment of the decision tree algorithm by [Morgan and Sonquist, 1963], many heuristic algorithms, such as ID3 [Quinlan, 1986], C4.5 [Quinlan, 1996] and CART [Breiman et al., 1984], are proposed to achieve the optimization target. However, these heuristic algorithms inevitably fall into sub-optimal solutions.

*corresponding author

A recent study based on a comprehensive benchmark shows that global solutions lead to average absolute improvements in test accuracy of 1 - 5% [Bertsimas and Dunn, 2017]. Several formulations and algorithms have been proposed to learn optimal decision trees through mixed integer programming (MIP) [Verwer and Zhang, 2017, 2019, Aghaei et al., 2020, Günlük et al., 2021], itemset mining [Nijssen and Fromont, 2007, Aglin et al., 2020], tailed branch-and-bound and dynamic programming [Hu et al., 2019, Lin et al., 2020, McTavish et al., 2022, Demirović et al., 2022] or SAT [Narodytska et al., 2018, Avellaneda, 2020, Ordyniak and Szeider, 2021, Hu et al., 2020, Shati et al., 2021]. However, a majority of these works can only address binary features. Binary encoding (converting continuous features into binary features) may lead to an enormous feature space, and may also lose information on the relationship between values of numerical features. Some of the aforementioned works have also proposed several approaches, such as bucketization [Verwer and Zhang, 2019], minimum description length principle [Demirović et al., 2022], and boosting tree based column elimination [McTavish et al., 2022], to deal with this problem.

Besides the works using binary-encoded features, the other research direction that focuses on directly addressing numerical features through MIP is rarely explored. Bertsimas and Dunn [2017] propose a seminal work on modelling the optimal classification tree (OCT) as a MIP problem, which directly address continuous features. This approach avoids the drawback of binary encoding and can be extended naturally to the multivariate (oblique) decision tree problem [Bertsimas and Dunn, 2019]. Recently, Zhu et al. [2020] present an SVM-based MIP formulation for learning optimal multivariate decision trees. This work also presents a sample selection method to select a subset of samples and use it to train the decision tree. Although the authors claim that the sample selection method can handle large datasets, the decision tree learned on the subset is not guaranteed to be the optimal one of the original dataset. The above MIP based approaches all rely on off-the-shelf MIP solvers such as CPLEX [Cplex, 2020] and Gurobi [Optimization, 2020], and have reported directly addressing datasets with up to 7,000 samples. Bertsimas and Dunn [2017] mentioned in their article that "datasets with tens of thousands of points were tested, but the time required for our methods to generate high-quality solutions was prohibitive". The main obstacle preventing the solvable problem from scaling to large datasets is that all these solvers implement a classical branch-and-bound (BB) scheme, which needs to branch on all binary variables to guarantee convergence, the dimension of which expands linearly with the size of the dataset.

In this paper, we propose a scalable deterministic global optimization algorithm for training optimal decision tree on classification task with numerical features. The first contribution of our work is that we reformulate the optimal decision tree training as a two-stage optimization problem. The stochastic programming community has made great progress in exploiting the structure of this type of problems [Karuppiyah and Grossmann, 2008, Li and Grossmann, 2019]. Recently, Cao and Zavala [2019] prove that for a two-stage problem with continuous second stage variables, branching only on the first stage variables can guarantee the convergence under mild assumptions. Many machine learning problems can be modeled as a two-stage problem and can apply this reduced-space BB method [Hua et al., 2021, Shi et al., 2022]. Although for the decision tree training problem, there are both binary and continuous variables in the second stage (i.e., sample specific variables, such as the predicted class of a sample), we prove that the convergence of our branch-and-bound algorithm can still be guaranteed by only branching on the first-stage variables (i.e., variables that represents the structure of the decision tree), the dimension of which is independent of the number of samples. Secondly, we provide several bounding strategies to construct the lower and upper bound of each BB node. Our new MIP formulation consists of a decomposable structure that can be directly parallelized when solving the lower bounding problem. We also present a tight and efficient MIP formula to obtain a lower bound quickly. Thirdly, we develop a sample reduction method that can predetermine the costs of some samples before the lower bound calculation at each BB node. Such a method can be combined with the lower bound calculation to further speed up the solution process, especially for large-scale datasets. The ablation test (Appendix D) performed on datasets reveals a consistent effectiveness on improving the computation speed of the BB process. Our experiments show that some BB nodes (e.g., when training the dataset BANKNOTE) can even have more than 99% of samples to be determined before computing the lower bound.

With these strategies, our tailored reduced-space BB algorithm can scale well on large datasets. For large datasets, the optimal solution found by our algorithm can on average reduce the optimal cost by 12.5% and improve the training accuracy by 3.6% and testing accuracy by 2.8%, compared to the current state-of-the-art [Bertsimas and Dunn, 2019]. Remarkably, our work is the first that

successfully trains a decision tree to global optimum on a dataset with 245,000 samples (1000 cores, less than 1% optimality gap, within 2 hours). We also improve the test accuracy by 1.7% on a dataset with almost one million samples (928,991 samples, 1000 cores, under 56.2% optimality gap within 4 hours), compared with CART and OCT. We highlight that we also report the optimality gap as a criteria to evaluate the goodness of the optimal solution, which is not presented in the majority of MIP based works. Bertsimas and Dunn [2019] states that the current MIP solvers struggle to provide a reasonable optimality gap for their OCT problem. It is also aligned with our numerical analysis that OCT still retains over 90% optimality gap on average for large datasets. At the same time, our algorithm can provide a practical gap on most of the dataset. Although OCT could find a better solution than CART at the early stage of the solving process, our results reveal that an even better solution could be discovered later and can result in better testing accuracy on large datasets.

In the following discussion of the paper, to avoid the ambiguity, we denote the node for the decision tree as *decision node* and *leaf node* and the node for the branch-and-bound algorithm as *BB node*.

2 Optimal Decision Tree with Decomposable Structure

In this work, we focus on training an optimal decision tree for the task of classification. Most part of the formulation presented here follows the work of Bertsimas and Dunn [2019]. However, in our formulation, we do not consider the constraint about the minimum number of samples required in the leaf node. Let $\mathcal{S} = \{1, \dots, n\}$ be the sample set and $\mathcal{K} = \{1, \dots, K\}$ be the class set. For a given data set $X = \{\mathbf{x}_s \mid \mathbf{x}_s \in \mathbb{R}^P, s \in \mathcal{S}\}$ with corresponding label set $Y = \{\mathbf{y}_s \mid \mathbf{y}_s \in \{0, 1\}^K, s \in \mathcal{S}\}$, where P is the number of features and K is the number of class. Initially, we scale each feature of the dataset to the range between 0 and 1. We seek to find an optimal decision tree model $F : \mathbb{R}^P \rightarrow \{0, 1\}^K$ with maximum depth D , such that:

$$\mathcal{L}(F) = \sum_{s \in \mathcal{S}} E(F(\mathbf{x}_s), \mathbf{y}_s) + \lambda R(F) \quad (1)$$

is minimized. Here, $E(\cdot)$ measures the misclassification error, $R(\cdot)$ is the regularization function that measure the complexity of the optimal decision tree model and λ is the complexity parameter. We denote $t \in \{1, \dots, T\}$ as the node of the tree, where $T = 2^{D+1} - 1$ is the size (total number of nodes) of the tree. Following the definition of Bertsimas and Dunn [2017], let $p(t) = \lfloor t/2 \rfloor$ be the parent decision node of t , $A_L(t)$ as the set of ancestors of t whose left branch is followed on the path from root node to node t , and $A_R(t)$ as the set of ancestors of t whose right branch is followed on the path from root node to node t . The tree node consists of two sets: *decision node* with $t \in \mathcal{T}_D = \{1, \dots, \lfloor T/2 \rfloor\}$ and *leaf node* with $t \in \mathcal{T}_L = \{\lfloor T/2 \rfloor + 1, \dots, T\}$. The training of optimal decision tree can be treated as an optimization problem (Problem 2):

$$\min_{a, b, c, d, z, L} \sum_{s \in \mathcal{S}} \left(\frac{1}{L} L_s + \frac{\lambda}{n} \sum_{t \in \mathcal{T}_D} d_t \right) \quad (2a)$$

$$\text{s.t. } \frac{1}{2} \sum_{k \in \mathcal{K}} (y_{sk} + c_{kt} - 2y_{sk}c_{kt}) - L_s \leq 1 - z_{st}, \forall t \in \mathcal{T}_L \quad (2b)$$

$$\sum_{k \in \mathcal{K}} c_{kt} = 1, \forall t \in \mathcal{T}_L \quad (2c)$$

$$\sum_{t \in \mathcal{T}_L} z_{st} = 1 \quad (2d)$$

$$\mathbf{a}_m^T (\mathbf{x}_s + \epsilon - \epsilon_{min}) + \epsilon_{min} \leq b_m + (1 + \epsilon_{max})(1 - z_{st}), \forall m \in A_L(t), t \in \mathcal{T}_L \quad (2e)$$

$$\mathbf{a}_m^T \mathbf{x}_s \geq b_m - (1 - z_{st}), \forall m \in A_R(t), t \in \mathcal{T}_L \quad (2f)$$

$$\sum_{j=1}^P a_{jt} = d_t, \forall t \in \mathcal{T}_D, j \in \{1, \dots, P\} \quad (2g)$$

$$0 \leq b_t \leq d_t, \forall t \in \mathcal{T}_D \quad (2h)$$

$$d_t \leq d_{p(t)}, \forall t \in \mathcal{T}_D \quad (2i)$$

$$0 \leq L_s \leq 1 \quad (2j)$$

$$a_{jt}, d_t \in \{0, 1\}, 0 \leq b_t \leq 1 \quad \forall t \in \mathcal{T}_D, j \in \{1, \dots, P\} \quad (2k)$$

$$z_{st}, c_{kt} \in \{0, 1\}, \quad \forall t \in \mathcal{T}_L \quad (2l)$$

$$s \in \mathcal{S} \quad (2m)$$

Here, variable a, b, c, d describe the structure of decision tree. Specifically, variable d_t on node $t \in \mathcal{T}_D$ determines whether a decision node splits or not. We track such split with variable $\mathbf{a}_t = [a_{1t}, \dots, a_{Pt}]^T \in \{0, 1\}^P$ and $b_t \in [0, 1]$. The prediction of each leaf node is controlled by the class indicator $c_{kt} \in \{0, 1\}, \forall t \in \mathcal{T}_L, k \in \mathcal{K}$. Variable $z_{st}, \forall t \in \mathcal{T}_L, s \in \mathcal{S}$ represents whether sample s falls into leaf t . Variable $L_s \in [0, 1]$ represents the loss of sample s . $y_{sk} \in \{0, 1\}$ is the element of vector $\mathbf{y}_s$ which represents whether sample s is labeled k . $\hat{L}$ is the parameter to normalize the misclassification with the baseline accuracy. ϵ, ϵ_{max} , and ϵ_{min} are constants to maintain the calculation stability of problem 2. Their values are generated following Bertsimas and Dunn [2019]. A detailed explanation of constraints can be found in Appendix A at supplementary materials.

3 Reduced-space Branch-and-Bound Algorithm

In this section, we model Problem 2 as a two-stage optimization problem and propose a reduced-space branch-and-bound (BB) algorithm with properly designed lower and upper bounds. Algorithm 1 (presented in Appendix B) elaborates the details of our algorithm. The algorithm starts from the initial feasible region M_0 . It narrows the gap between the lower and upper bound by partitioning M_0 recursively into smaller regions and pruning any regions that are verified to contain no optimal solution. In each iteration with particular sub-region M , lower bound $\beta(M)$ and upper bound $\alpha(M)$ are computed. If the gap between β and α is smaller than ϵ , Algorithm 1 terminates. Otherwise M is partitioned into two disjoint sub-region with some branching strategies explained in Appendix B.2.

A critical feature of our algorithm is that it can ensure the convergence by only branching on the first stage variables (i.e. a, b, c, d). Cao and Zavala [2019] prove that for a two-stage optimization problem, if the second stage variables are continuous, then the BB algorithm can converge by branching only on the first stage variables. Although the decision tree training problem is different since we have both continuous and binary second stage variables, we can still prove its convergence.

Theorem 1. *Algorithm 1 converges to the global optimum, in the sense that*

$$\lim_{i \rightarrow \infty} \alpha_i = \lim_{i \rightarrow \infty} \beta_i = f \quad (3)$$

The proof of Theorem 1 is presented in Appendix B.1.

3.1 Two Stage Optimization Problem

Note that variables a, b, c, d describe the structure of decision tree and are the same for all sample, while variables z and L are sample-specific variables and describe the allocation and cost of a specific sample. Therefore, Problem 2 can be reformulated as a two-stage optimization problem in the following form:

$$f(M_0) = \min_{m \in M_0} \sum_{s \in \mathcal{S}} Q_s(m) \quad (4)$$

where we denote $m = (a, b, c, d)$ as all first-stage variables and $Q_s(\cdot)$ represents the optimal value of the second-stage problem:

$$Q_s(m) = \min_{z_s, L_s} \frac{1}{\hat{L}} L_s + \frac{\lambda}{n} \sum_{t \in \mathcal{T}_D} d_t \quad (5a)$$

$$\text{s.t. Constraint 2b} - 2l \quad (5b)$$

The closed set $M_0 := [m^l, m^u]$ denotes the region of the first stage variables. $(\cdot)^l, (\cdot)^u$ represent the lower and upper bound of each variable. In each BB node with a specific partition $M \subseteq M_0$, we solve the problem $f(M) = \min_{m \in M} \sum_{s \in \mathcal{S}} Q_s(m)$.

3.2 Lower Bounding Problem

Basic lower bounding problem: the problem we solve at each BB node has an implicit constraint. That is, all samples share the same first stage variables (i.e., tree structure). This is called non-anticipativity constraints in the stochastic programming community. We can obtain a basic lower bound by relaxing this constraint and solve the resulting problem:

$$\beta(M) := \min_{m_s \in M} \sum_{s \in \mathcal{S}} Q_s(m_s) \quad (6)$$

such a problem can be easily decomposed into n subproblems $\beta_s(M) := \min_{m \in M} Q_s(m)$ with $\beta(M) := \sum_{s \in \mathcal{S}} \beta_s(M)$. The optimal value of β_s can be calculated through Algorithm 2 (presented in Appendix C) by enumerating all possible leaf nodes that sample s can fall into, without the need to solve any optimization problems explicitly. The complexity of Algorithm 2 is $O(|\mathcal{T}_D| + |\mathcal{T}_L|)$. Since the depth of the tree is typically small for the sake of interpretability, the speed of Algorithm 2 can be executed much faster than other methods.

Theorem 2. *With given bound $M = [m^l, m^u]$, Algorithm 2 finds all possible leaf nodes that sample s can fall into and gives the global optimal value of $\beta_s(M)$.*

The proof of Theorem 2 is in Appendix C.1.

Relaxed MIP problem: One byproduct of Algorithm 2 is $\mathcal{T}_{z_s} \subseteq \mathcal{T}_L$, the set of all possible leaf nodes that sample s can fall into. Figure 1 in Section 4 provides an example on the determination of $\mathcal{T}_{z_s}$. We have $z_{st} = 0, \forall t \notin \mathcal{T}_{z_s}$. For each sample, basic lower bounding method computes the lower bound by comparing the label y_{sk} with the prediction c_{kt} of each leaf node in $t \in \mathcal{T}_{z_s}$. Note here basic lower bounding method relaxes the non-anticipativity constraints on c and allow different values of c for each sample. We can enforce this constraint to generate a tighter lower bound by solving the following MIP problem:

$$\beta^R(M) = \min_{c, L} \sum_{s \in \mathcal{S}} \frac{1}{L} L_s + \lambda \sum_{t \in \mathcal{T}_D} d_t^l \quad (7a)$$

$$\text{s.t. } L_s \geq 1 - \sum_{t \in \mathcal{T}_{z_s}} \sum_{k \in \mathcal{K}} y_{sk} c_{kt} \quad (7b)$$

$$\sum_{k \in \mathcal{K}} c_{kt} = 1, \quad \forall t \in \mathcal{T}_L \quad (7c)$$

$$0 \leq L_s \leq 1 \quad (7d)$$

$$c_{kt} \in \{0, 1\}, \quad \forall t \in \mathcal{T}_L, k \in \mathcal{K} \quad (7e)$$

$$s \in \mathcal{S} \quad (7f)$$

where d^l is the lower bound of variable d . Since the only number of integer variables in Problem 7 is c , the dimension of which is typically small and independent of the number of samples, solving Problem 7 is much easier than the original problem. Because Problem 7 is based on the $\mathcal{T}_{z_s}$ calculated from the basic lower bounding method but enforces non-anticipativity constraints on c , we have:

Proposition 3. $\beta(M) \leq \beta^R(M) \leq f(M)$

Notice that when the variable c is fixed ($c^l = c^u$ in M) at a BB node, $\beta(M) = \beta^R(M)$ holds and we do not need to compute $\beta^R(M)$.

Group decomposition: for the basic lower bounding method, we treat each sample as one subproblem. One natural extension is to assign multiple samples to a subproblem. Specifically, we define the group set $\mathcal{G} = \{1, \dots, G\}$, and we partition $\mathcal{S}$ into G groups $\{\mathcal{S}_1, \dots, \mathcal{S}_G \mid \bigcup_{g=1}^G \mathcal{S}_g = \mathcal{S} \text{ and } \mathcal{S}_i \cap \mathcal{S}_g = \emptyset, \forall i, g \in \mathcal{G}, i \neq g\}$, then we can obtain a lower bound from:

$$\beta^G(M) := \min_{m_g \in M} \sum_{g \in \mathcal{G}} Q_g(m_g), \quad (8)$$

which can be decomposed into G subproblems with $\beta^G(M) = \sum_{g \in \mathcal{G}} \beta_g^G(M)$ and $\beta_g^G(M) := \min_{m_g \in M} Q_g(m_g)$. Here each subproblem g can be obtained by replacing $s \in \mathcal{S}$ in the original problem with $s \in \mathcal{S}_g$. The non-anticipativity constraints is still enforced within each subproblem (i.e. a, b, c, d are the same for all samples within the group), while the non-anticipativity constraints between groups are relaxed. Hence, the β^G provides a tighter lower bound than the basic lower bound β .

Proposition 4. $\beta(M) \leq \beta^G(M) \leq f(M)$

The fewer groups we split, the tighter is the lower bound. However, the solution time of each subproblem is also longer. Therefore, there is a trade-off between the efficiency and tightness when setting the number of groups. Besides this, the way of assigning samples to groups also influences the quality of lower bounds. Finding the best grouping scheme is itself NP-hard. In our implementation, we adopted the heuristic results (e.g. CART) to navigate the assignment of groups. First, we divide all samples into raw groups based on leaves they fall under from the heuristic solution. Second, in each raw group, a k -means clustering is launched. Finally, the samples for each cluster at each raw group are evenly assigned to G groups. In this way, samples are approximately evenly distributed among different groups. We keep the group assignment the same for all BB nodes.

3.3 Upper Bounding Problem

The upper bound can be easily obtained by choosing any arbitrary first stage solution $\hat{m} \in M$. Let $\alpha(M)$ be the upper bounding problem, we have $\alpha(M) = \sum_{s \in \mathcal{S}} Q_s(\hat{m})$. The calculation of $\alpha(M)$ is the same as the evaluation of a decision tree $\hat{m}$, which is computationally cheap. In our implementation, at the root BB node, we use CART to provide a feasible solution $\hat{m}$ for the initial upper bound. We also propose two heuristics in each descendent BB node to enable a fast search for an optimal solution. First, the optimal solution to each subproblem directly provides a feasible solution. We calculate the corresponding cost and pick the most optimal one to be the upper bound. This method does not require additional computational time to generate $\hat{m}$. However, it could be affected by the group assignment. The second method applies bootstrapping to randomly generate a small subset of samples and train a decision tree for the subset. This "bootstrapped" subproblem provides a feasible solution and introduces more possibilities for finding a better upper bound.

4 Sample Reduction

During the calculation of the basic lower bound, Algorithm 2 reduces the possible leaf nodes that sample s can fall into from $\mathcal{T}_L$ to $\mathcal{T}_{z_s}$. Comparing the label y_{sk} with the range of c of each leaf node in $t \in \mathcal{T}_{z_s}$, the loss of some samples can be determined. Specifically, suppose k is the label of sample s (i.e. $y_{sk} = 1$). Equation 9 checks the loss of sample s under M .

$$L_s = \begin{cases} 1 & \text{if } \bigwedge_{t \in \mathcal{T}_{z_s}} c_{kt}^l = c_{kt}^u = 0 \\ 0 & \text{if } \bigwedge_{t \in \mathcal{T}_{z_s}} c_{kt}^l = c_{kt}^u = 1 \\ undtm. & \text{otherwise} \end{cases} \quad (9)$$

Figure 1 provides an example to illustrate the determination of $\mathcal{T}_{z_s}$ and the loss determination of a sample s under some conditions of c^l, c^u . If the loss of sample s is determined at the current BB node, we no longer need to take this sample into consideration in the lower bound calculation, resulting in sample reduction. Moreover, determined loss holds for all descendent BB nodes. Therefore, the determination of some sample's loss can significantly reduce the computational load, especially when the level of the search tree goes deep. With sample reduction, many of these samples can determine their loss without being involved in the solution of any optimization problem. Indeed, for dataset SKIN-SEGMENTATION (one of the largest test dataset in our experiment with $n = 245,057$), on average, 63.6% of the samples can be determined among all BB nodes. For some BB nodes, 99.6% of samples can have their loss determined. Such a strategy significantly improves node exploration efficiency.

We notice that a similar "data selection" method is proposed in [Zhu et al., 2020]. A major difference between their method and our "cost determination" method is that the solution found from the

"selected" data subset can not guarantee the global optimum of the original dataset. In contrast, in our method, the loss of removed samples is not deleted but pre-determined and added to the final total loss calculation. Hence, the optimal solution we obtain is the same as the optimal solution of the original problem.

Improvements on the calculation of lower bound: define the sample set with determined loss as $\mathcal{S}_{dt}$ and those with undetermined loss as $\mathcal{S}_{ud}$, such that $|\mathcal{S}_{dt}| + |\mathcal{S}_{ud}| = n$. Here, we analyze the effects of sample reduction combined with the reach-leaf determination ($\mathcal{T}_{z_s}$) on three lower bound strategies. Initially, the calculation of the basic lower bounding problem requires running Algorithm 2 for n times. With sample reduction, the basic lower bounding problem (Problem 6) can be rewritten as:

$$\beta(M) := \min_{m_s \in M} \sum_{s \in \mathcal{S}_{ud}} Q_s(m_s) + \sum_{s \in \mathcal{S}_{dt}} \frac{1}{L} L_s \quad (10)$$

Here, $L_s, \forall s \in \mathcal{S}_{dt}$ is already determined at the parent BB node via the sample reduction. Thus, the computing load of Problem 6 can be reduced from $O(n)$ to $O(|\mathcal{S}_{ud}|)$. Since M of a descendent BB node is a subset of its parent BB node, $\mathcal{T}_{z_s}$ can also be inherited from the parent BB node, and the calculation of $\mathcal{T}_{z_s}$ only need to be executed on samples in $\mathcal{S}_{ud}$. Similarly, for the lower bound from relaxed MIP, the number of samples involved in the optimization is reduced from n to $|\mathcal{S}_{ud}|$. Finally, for the group decomposition based lower bounding strategy, the number of samples involved in each subproblem is reduced from $|\mathcal{S}_g|$ to $|\mathcal{S}_{ud} \cap \mathcal{S}_g|$. The detailed formulations of these lower bounding methods after sample reduction can be found in Appendix D.

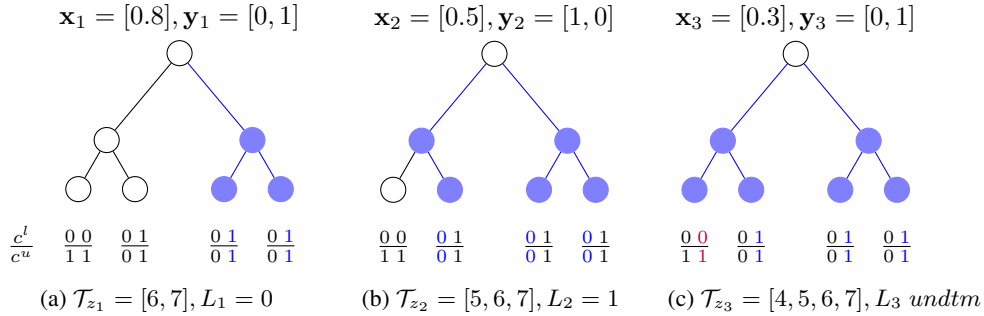


Figure 1: Loss determination under the bounds $a^l = [1, 1, 1]$, $a^u = [1, 1, 1]$, $b^l = [0.2, 0.1, 0.1]$, $b^u = [0.6, 0.4, 1.0]$, $c^l = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}$, $c^u = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$. The nodes in blue indicate the possible paths of a sample under current bound. The bound of c for each leaf t is denoted as $\frac{c^l}{c^u}$ under each leaf node. **Sample 1:** since $\mathbf{x}_1[1] \geq b_1^u$, $\mathbf{x}_1$ can only fall into the right descendent of the root decision node, and as $\mathbf{x}_1[1] \in [b_3^l, b_3^u]$, leaf node 3 and 4 can be reached by sample 1 and thus $\mathcal{T}_{z_1} = \{6, 7\}$. Since $c_{2t}^l = c_{2t}^u = 1, \forall t \in \mathcal{T}_{z_1}$ (colored in blue), which is identical to y_{12} , we have $L_1 = 0$. **Sample 2:** since $\mathbf{x}_2[1] \in [b_1^l, b_1^u]$, $\mathbf{x}_2$ can fall into both descendent of the root decision node. However, as $\mathbf{x}_2[1] \geq b_2^u$, leaf node 1 can not be reached by sample 2, and thus $\mathcal{T}_{z_2} = \{5, 6, 7\}$. Since y_2 is inconsistent with all the $[c_t^l, c_t^u], \forall t \in \mathcal{T}_{z_2}$, we determine that $L_2 = 1$. **Sample 3:** since $\mathbf{x}_3[1] \in [b_1^l, b_1^u], \forall t \in \mathcal{T}_D$, all leaf node can be reached by sample 3, $\mathcal{T}_{z_3} = \{4, 5, 6, 7\}$. Sample 3 can have several possible loss values if fall into different leaf node in $\mathcal{T}_{z_3}$. Here, variable c is fixed in all leaves except leaf 1 (colored in red). Therefore, we are unable to determine L_3 .

5 Computational Experiments

We evaluate the performance of our algorithm (denoted as RS-OCT) on 21 real-world datasets from UCI machine learning repository [Lichman et al., 2013]. The algorithm is implemented in Julia 1.7.0. Our implementation code is available at: https://github.com/YankaiGroup/optimal_decision_tree. Comparison is made against the CART algorithm [Breiman et al., 1984] and Optimal Classification Tree (OCT) method [Bertsimas and Dunn, 2017]. A comparison with the performance of FlowOCT [Aghaei et al., 2020] is also presented in Appendix E.1. To avoid the effects of hyper-parameter choice, the penalty parameter λ is set to 0.05 for all three algorithms. The split of training and testing data follows the work of Bertsimas and Dunn [2017], that is, 75% of the entire dataset is selected as the training set while the rest 25% is set as the testing set. All datasets are

split by the `stratifiedobs` function from Julia Package `MLDataUtils`, under random seed 1, so that the distribution of class is guaranteed to be the same in both training and testing set. RS-OCT and OCT are executed until either they converge to an optimality gap within 1% or the runtime exceeds a limit of 4 hours. The result of CART is obtained through the Julia Package `DecisionTree` with version v0.10.12. The MIP problem of OCT and the subproblem of RS-OCT (for grouping based lower bound) are solved by the state-of-art global optimizer CPLEX 20.1.0 [Cplex, 2020]. Both OCT and RS-OCT are provided with the same CART solution as warm-start. A preliminary parallel version of our algorithm is implemented by assigning subproblems in the lower bound calculation to multiple CPU cores. The size of each subproblem for group decomposition is set as $\max(2K, \frac{2^D n}{(ct+P+2)-2^D(P+2+K)})$, where ct (default: 150) is a constant that controls the average group-size. The serial and parallel experiments are executed on a high-performance computing cluster (40 Intel "Skylake" at 2.4 GHz with 202G RAM per computing node). Five criteria (upper bound, training accuracy, testing accuracy, optimality gap, and time) are used to compare the performance of the algorithms. Here, the upper bound (UB) represents the best feasible solution we found until the termination of each algorithm. The optimality gap is calculated as $\frac{UB-LB}{UB} \times 100\%$. It measures the closeness from the best-searched solution (i.e. UB) to its possible global optimum (i.e. lower bound). As a unique property of deterministic global optimization algorithms, heuristic algorithms like CART cannot provide such criteria. Our work focuses on improving the solution process of the optimal decision tree problem. Therefore, to best represent the solving performance, we use the same value of λ , so that both our method and OCT are solving the same optimization problem, and we can have a fair comparison of the optimality gap, solution time, and best achieved upper bound on this specific problem formulation. However, to further demonstrate the performance of RS-OCT in practical, compared to other state-of-the-art algorithms, we also perform experiments with hyperparameter tuning on depth 2 and 3. The results are presented in Appendix E.1. The effectiveness of sample reduction is validated via the ablation test and the result is shown in Appendix E.2.

Small and medium-scale datasets We perform experiments on 15 datasets ranging from 210 to 5,473 samples. Table 1 presents the result of using three algorithms. In terms of UB, RS-OCT can find a better (or equally better) solution with the lowest cost for almost all datasets, compared to CART and OCT. RS-OCT also terminates with a smaller optimality gap for all datasets. It is remarkable that for datasets such as GLASS, BANKNOTE, and WALL-FOLLOWING, RS-OCT can achieve an optimality gap within 1% while OCT still has a large gap over 98% after 4 hours of running. Our result indicates that although OCT could find a better solution than CART, it is still hard to claim that such a solution is the optimal one. In contrast, RS-OCT can find an even better solution more efficiently and prove the optimality within a reasonable time. Due to the better cost found by RS-OCT, the training accuracy of all datasets in Table 1 improves by 2.9% on average compared to CART and 0.8% on average compared to OCT. In terms of testing accuracy, since we fixed the penalty parameter λ for all datasets, the result on smaller datasets ($n < 1000$) is unsatisfactory probably due to an overfitting issue. In total, the testing accuracy is improved by 2.2% and 1.0%, on average, compared to CART and OCT, respectively. For medium-scale datasets ($n \geq 1000$), the testing accuracy improves by 3.3% on average compared to CART and 1.4% on average compared to OCT.

Large-scale datasets We test our algorithm on several large datasets ($n \geq 7,000$), with a variety of dimensions (3-16) and number of classes (2-12), as shown in Table 2. RS-OCT is executed in parallel with 1000 CPU cores. Remarkably, RS-OCT outperforms OCT and CART on all criteria for all datasets, including the HT-SENSOR which has almost one million samples. In terms of upper bound, we notice that for datasets with a size over 40,000 samples, OCT can not provide a better solution than the warm-start given by CART. Nevertheless, RS-OCT discovers better solutions in terms of UB than both CART (12.5% improvement on average) and OCT (11.7% improvement on average) on all large datasets. In terms of optimality gap, RS-OCT converges to an optimality gap within 1% on PENDIGITS, SHUTTLE, and SKIN-SEGMENTATION. It verifies that the best solutions found for these datasets are merely at most 1% close to the actual global optimal solution. It is the first time an optimal decision tree (without binary encoding) is found for a dataset with almost **one million** samples with a meaningful optimality gap (56.2%) in four hours of runtime. Remarkably, our algorithm can also solve for a dataset with over 200,000 samples to a practical optimality gap ($\leq 1\%$) in less than two hours. Given the more optimal solution found by RS-OCT, its training accuracy is on average boosted by 3.7% and 3.6%, compared to CART and OCT, respectively. The testing accuracy

Table 1: Numerical results on small and median datasets ($D = 2$, $\lambda = 0.05$, serial). The results of testing accuracy reported in [Bertsimas and Dunn, 2017] are also listed in the paranthesis for CART and OCT. These "paranthesis" results are obtained within the runtime of 2 hours.

DATA-SET	n	P	K	METHOD	UB	TRAINING ACCURACY(%)	TESTING ACCURACY(%)	GAP (%)	TIME (s)
SEEDS	210	7	3	CART	42.2	91.2	94.1 (87.2)	-	-
				OCT	27.2	94.3	86.3 (88.7)	<1%	579.1
				RS-OCT	27.2	94.3	88.2	<1%	151.2
GLASS	214	9	6	CART	152.2	65.8	64.3	-	-
				OCT	143.8	67.7	64.3	99.9	14400
				RS-OCT	140.9	68.4	62.5	<1%	2503.8
BODY	507	5	2	CART	74.3	90.0	88.9	-	-
				OCT	58.7	92.1	92.1	<1%	10985
				RS-OCT	58.7	92.1	92.1	<1%	965.9
STATLOG-GERMAN	1,000	24	2	CART	281.6	73.7	71.2 (70.1)	-	-
				OCT	280.2	73.9	71.2 (70.5)	99.9	14400
				RS-OCT	267.3	75.1	72.0	38.5	14400
CONCRETE	1,030	8	3	CART	667.6	62.6	61.9	-	-
				OCT	621.4	65.2	65.4	100	14400
				RS-OCT	621.4	65.2	63.0	13.0	14400
BANKNOTE	1,372	4	2	CART	185.6	90.0	89.0 (89.0)	-	-
				OCT	155.0	91.6	91.0 (90.1)	98.7	14400
				RS-OCT	135.2	92.7	92.2	<1%	675.6
CONTRA-CEPTIVE	1,473	11	3	CART	1353.7	47.6	46.3 (46.8)	-	-
				OCT	1215.6	53.0	51.0 (48.4)	54.5	14400
				RS-OCT	1189.8	54.0	56.4	12.0	14400
OZONE-EIGHT	1,847	72	2	CART	103.3	93.1	93.1 (93.1)	-	-
				OCT	102.2	93.2	92.8 (93.1)	100	14400
				RS-OCT	94.7	93.7	92.4	97.6	14400
OZONE-ONE	1,848	72	2	CART	43.5	97.0	96.8 (96.8)	-	-
				OCT	39.3	97.3	96.8 (96.8)	99.9	14400
				RS-OCT	40.4	97.2	96.8	99.6	14400
THYROID-ANN	3,772	21	3	CART	140.6	95.4	95.6 (95.6)	-	-
				OCT	86.7	97.2	96.8 (95.6)	99.9	14400
				RS-OCT	66.1	97.8	98.1	65.4	14400
STATLOG-LANSAT	4,435	36	6	CART	4861.3	64.7	64.6 (63.2)	-	-
				OCT	4786.8	65.2	64.8 (63.2)	100	14400
				RS-OCT	4757.8	65.4	66.0	40.1	14400
SPAMBASE	4,601	57	2	CART	826.9	85.5	84.5 (84.2)	-	-
				OCT	826.9	85.5	84.5 (84.3)	100	14400
				RS-OCT	756.0	86.7	85.9	81.4	14400
WALL-FOLLOWING	5,456	2	4	CART	608.8	94.0	94.0 (94.0)	-	-
				OCT	608.8	94.0	94.0 (94.0)	100	14400
				RS-OCT	608.8	94.0	94.0	<1%	223.2
PAGE-BLOCK	5,473	10	5	CART	315.4	93.1	93.2	-	-
				OCT	273.1	93.9	93.8	100	14400
				RS-OCT	209.6	95.4	95.0	46.2	14400

is also boosted by 3.3% and 2.8% on CART and OCT, respectively. Notably, compared with the results from Table 1, we find that the performance of RS-OCT is improving with the increment of the data size.

6 Conclusion

This paper proposed a tailed reduced-space branch and bound (BB) algorithm to train optimal decision tree for the classification tasks. Our BB algorithm can converge to a global ϵ -optimal solution by only branching on variables describing the tree structure, which is invariant to the number of samples. Combined with the sample reduction and paralleled lower bound calculation, our algorithm can perform well on all datasets in terms of UB, optimality gap and training accuracy. Our algorithm also improves the testing accuracy on all large datasets ($n \geq 7,000$).

Impact and Future work: We hope that the MIP-based scalable framework proposed in this paper can help address the needs of optimal decision tree for large-scale data training in high-stake domains such as crime analysis [Zhuang et al., 2017], or medical decision [Bertsimas et al., 2018]. We look forward to integrating our algorithms with other functionality (e.g., fairness [Aghaei et al., 2019]) to provide handy tools for wider applications.

Table 2: Numerical results on large datasets ($D = 2, \lambda = 0.05$, paralleled with 1000 cores). Bertsimas and Dunn [2017] do not report results on datasets of this scale.

DATA-SET	n	P	K	METHOD	UB	TRAINING ACCURACY(%)	TESTING ACCURACY(%)	GAP (%)	TIME (s)
PENDIGITS	7,494	16	10	CART	32993.0	38.9	39.1	-	-
				OCT	32993.0	38.9	39.1	100	14400
				RS-OCT	32426.1	40.0	39.9	<1%	2093.5
AVILA	10,430	10	12	CART	9454.3	50.4	51.0	-	-
				OCT	9196.4	51.7	52.5	100	14400
				RS-OCT	8787.5	53.8	53.8	19.6	14400
EEG	14,980	14	2	CART	7815.8	61.7	61.2	-	-
				OCT	7748.7	61.7	61.0	100	14400
				RS-OCT	6752.7	66.9	65.5	50.8	14400
HTRU	17,898	8	2	CART	327.0	97.8	97.7	-	-
				OCT	320.5	97.8	97.7	100	14400
				RS-OCT	300.7	98.0	97.8	42.8	14400
SHUTTLE	43,500	9	7	CART	2567.5	93.8	93.8	-	-
				OCT	2567.5	93.8	93.8	100	14400
				RS-OCT	1908.1	95.4	95.5	<1%	586.1
SKIN-SEGMENTATION	245,057	3	2	CART	23409.5	89.9	89.8	-	-
				OCT	23409.5	89.9	89.8	100	14400
				RS-OCT	16953.7	92.7	92.7	<1%	6698.9
HT-SENSOR	928,991	11	3	CART	782046.1	58.1	58.1	-	-
				OCT	782046.1	58.1	58.1	100	14400
				RS-OCT	753075.8	59.7	59.7	56.2	14400

Limitation: Similar to other works, our algorithm only tests optimal decision tree problems with small depth (e.g. $D = 2, 3$). Larger D slows down the algorithm’s efficiency. Proper bound tightening methods will be designed in the future to solve this problem.

Acknowledgments and Disclosure of Funding

Y.C. acknowledges funding from the discovery program of the Natural Science and Engineering Research Council of Canada under grant RGPIN-2019-05499. The authors also gratefully acknowledge the computing resources provided by SciNet (www.scinethpc.ca) and Digital Research Alliance of Canada (www.alliancecan.ca).

References

- Sina Aghaei, Mohammad Javad Azizi, and Phebe Vayanos. Learning optimal and fair decision trees for non-discriminative decision-making. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1418–1426, 2019.
- Sina Aghaei, Andres Gomez, and Phebe Vayanos. Learning optimal classification trees: Strong max-flow formulations. *arXiv preprint arXiv:2002.09142*, 2020.
- Gaël Aglin, Siegfried Nijssen, and Pierre Schaus. Learning optimal decision trees using caching branch-and-bound search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3146–3153, 2020.
- Florent Avellaneda. Efficient inference of optimal decision trees. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3195–3202, 2020.
- Dimitris Bertsimas and Jack Dunn. Optimal classification trees. *Machine Learning*, 106(7):1039–1082, 2017.
- Dimitris Bertsimas and Jack Dunn. *Machine learning under a modern optimization lens*. Dynamic Ideas LLC, 2019.
- Dimitris Bertsimas, Jack Dunn, Colin Pawlowski, John Silberholz, Alexander Weinstein, Ying Daisy Zhuo, Eddy Chen, and Aymen A Elfiky. Applied informatics decision support tool for mortality predictions in patients with cancer. *JCO clinical cancer informatics*, 2:1–11, 2018.
- L Breiman, JH Friedman, R Olshen, and CJ Stone. Classification and regression trees. 1984.

- Yankai Cao and Victor M Zavala. A scalable global optimization algorithm for stochastic nonlinear programs. *Journal of Global Optimization*, 75(2):393–416, 2019.
- IBM ILOG Cplex. V20.1.0: User’s manual for CPLEX. *International Business Machines Corporation*, 2020.
- Emir Demirović, Anna Lukina, Emmanuel Hebrard, Jeffrey Chan, James Bailey, Christopher Leckie, Kotagiri Ramamohanarao, and Peter J Stuckey. Murtree: Optimal decision trees via dynamic programming and search. *Journal of Machine Learning Research*, 23(26):1–47, 2022.
- Oktay Günlük, Jayant Kalagnanam, Minhan Li, Matt Menickelly, and Katya Scheinberg. Optimal decision trees for categorical data via integer programming. *Journal of global optimization*, 81(1): 233–260, 2021.
- Reiner Horst and Hoang Tuy. *Global optimization: Deterministic approaches*. Springer Science & Business Media, 2013.
- Hao Hu, Mohamed Siala, Emmanuel Hebrard, and Marie-José Huguet. Learning optimal decision trees with maxsat and its integration in adaboost. In *IJCAI-PRICAI 2020, 29th International Joint Conference on Artificial Intelligence and the 17th Pacific Rim International Conference on Artificial Intelligence*, 2020.
- Xiyang Hu, Cynthia Rudin, and Margo Seltzer. Optimal sparse decision trees. *Advances in Neural Information Processing Systems*, 32, 2019.
- Kaixun Hua, Mingfei Shi, and Yankai Cao. A scalable deterministic global optimization algorithm for clustering problems. In *International Conference on Machine Learning*, pages 4391–4401. PMLR, 2021.
- Ramkumar Karuppiah and Ignacio E Grossmann. A lagrangean based branch-and-cut algorithm for global optimization of nonconvex mixed-integer nonlinear programs with decomposable structures. *Journal of global optimization*, 41(2):163–186, 2008.
- Hyafil Laurent and Ronald L Rivest. Constructing optimal binary decision trees is np-complete. *Information processing letters*, 5(1):15–17, 1976.
- Can Li and Ignacio E Grossmann. A generalized benders decomposition-based branch and cut algorithm for two-stage stochastic programs with nonconvex constraints and mixed-binary first and second stage variables. *Journal of Global Optimization*, 75(2):247–272, 2019.
- Moshe Lichman et al. Uci machine learning repository, 2013.
- Jimmy Lin, Chudi Zhong, Diane Hu, Cynthia Rudin, and Margo Seltzer. Generalized and scalable optimal sparse decision trees. In *International Conference on Machine Learning*, pages 6150–6160. PMLR, 2020.
- Hayden McTavish, Chudi Zhong, Reto Achermann, Ilias Karimalis, Jacques Chen, Cynthia Rudin, and Margo Seltzer. Fast sparse decision tree optimization via reference ensembles. In *AAAI Conference on Artificial Intelligence*, 2022.
- James N Morgan and John A Sonquist. Problems in the analysis of survey data, and a proposal. *Journal of the American statistical association*, 58(302):415–434, 1963.
- Nina Narodytska, Alexey Ignatiev, Filipe Pereira, Joao Marques-Silva, and IS RAS. Learning optimal decision trees with sat. In *Ijcai*, pages 1362–1368, 2018.
- Siegfried Nijssen and Elisa Fromont. Mining optimal decision trees from itemset lattices. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 530–539, 2007.
- Gurobi Optimization. Gurobi optimizer 9.0 reference manual. 2020.
- Sebastian Ordyniak and Stefan Szeider. Parameterized complexity of small decision tree learning. *Cancer*, 683(90):18, 2021.

- J. Ross Quinlan. Induction of decision trees. *Machine learning*, 1(1):81–106, 1986.
- J Ross Quinlan. Improved use of continuous attributes in c4.5. *Journal of artificial intelligence research*, 4:77–90, 1996.
- Pouya Shati, Eldan Cohen, and Sheila McIlraith. Sat-based approach for learning optimal decision trees with non-binary features. In *27th International Conference on Principles and Practice of Constraint Programming (CP 2021)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2021.
- Mingfei Shi, Kaixun Hua, Jiayang Ren, and Yankai Cao. Global optimization of k-center clustering. In *International Conference on Machine Learning*, pages 19956–19966. PMLR, 2022.
- Sicco Verwer and Yingqian Zhang. Learning decision trees with flexible constraints and objectives using integer optimization. In *International Conference on AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems*, pages 94–103. Springer, 2017.
- Sicco Verwer and Yingqian Zhang. Learning optimal classification trees using a binary linear program formulation. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 1625–1632, 2019.
- Haoran Zhu, Pavankumar Murali, Dzung Phan, Lam Nguyen, and Jayant Kalagnanam. A scalable mip-based method for learning optimal multivariate decision trees. *Advances in Neural Information Processing Systems*, 33:1771–1781, 2020.
- Yong Zhuang, Matthew Almeida, Melissa Morabito, and Wei Ding. Crime hot spot forecasting: A recurrent model with spatial and temporal information. In *2017 IEEE International Conference on Big Knowledge (ICBK)*, pages 143–150. IEEE, 2017.

Checklist

1. For all authors...
 - (a) Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope? [\[Yes\]](#)
 - (b) Did you describe the limitations of your work? [\[Yes\]](#) See section 6
 - (c) Did you discuss any potential negative societal impacts of your work? [\[N/A\]](#) Our work is focus on developing novel training method on optimal decision tree
 - (d) Have you read the ethics review guidelines and ensured that your paper conforms to them? [\[Yes\]](#)
2. If you are including theoretical results...
 - (a) Did you state the full set of assumptions of all theoretical results? [\[Yes\]](#)
 - (b) Did you include complete proofs of all theoretical results? [\[Yes\]](#) all proofs are in appendix file from the supplementary materials
3. If you ran experiments...
 - (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [\[Yes\]](#)
 - (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [\[Yes\]](#) See Section 5
 - (c) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [\[N/A\]](#) Since our algorithm is a deterministic global optimization algorithm, whatever random seed provide, the algorithm will always converge to the same global optimum (with given optimality gap)
 - (d) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [\[Yes\]](#) See Section 5
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
 - (a) If your work uses existing assets, did you cite the creators? [\[Yes\]](#) OCT is coded by ourselves and the code is provided in the supplementary materials and we also provide the result reported in their original paper for reference.

- (b) Did you mention the license of the assets? [\[Yes\]](#)
 - (c) Did you include any new assets either in the supplemental material or as a URL? [\[Yes\]](#)
 - (d) Did you discuss whether and how consent was obtained from people whose data you're using/curating? [\[N/A\]](#) We are using data from UCI Machine Learning Repository
 - (e) Did you discuss whether the data you are using/curating contains personally identifiable information or offensive content? [\[N/A\]](#) We are using data from UCI Machine Learning Repository
5. If you used crowdsourcing or conducted research with human subjects...
- (a) Did you include the full text of instructions given to participants and screenshots, if applicable? [\[N/A\]](#) Our work didn't use crowdsourcing or conducted research with human subjects
 - (b) Did you describe any potential participant risks, with links to Institutional Review Board (IRB) approvals, if applicable? [\[N/A\]](#) Our work didn't use crowdsourcing or conducted research with human subjects
 - (c) Did you include the estimated hourly wage paid to participants and the total amount spent on participant compensation? [\[N/A\]](#) Our work didn't use crowdsourcing or conducted research with human subjects

Supplementary Materials

A Constraint Explanation for Problem 2

Constraint 2b checks the loss of each sample and the derivation is shown as follows.

Let a latent variable L_{st} to be the loss of sample s at leaf node t , as y_{sk} and c_{kt} are both binary, we can set:

$$L_{st} = \frac{1}{2} \sum_{k \in \mathcal{K}} (y_{sk} - c_{kt})^2 \quad (11a)$$

$$= \frac{1}{2} \sum_{k \in \mathcal{K}} (y_{sk} + c_{kt} - 2y_{sk}c_{kt}), \quad \forall t \in \mathcal{T}_L, s \in \{1, \dots, n\} \quad (11b)$$

It is easy to see that $0 \leq L_{st} \leq 1$, as the loss of each sample s at leaf t can only have one mis-match on class label. We introduce variable $L_s \in [0, 1]$ to indicate the loss of sample s in its best allocated leaf. To guarantee that the best leaf node for sample s is chosen, we conclude the following big-M constraints:

$$-M(1 - z_{st}) \leq L_s - L_{st} \quad (12)$$

As $L_s - L_{st}$ will no exceed 1, combined with Equation 11, we have:

$$\frac{1}{2} \sum_{k \in \mathcal{K}} (y_{sk} + c_{kt} - 2y_{sk}c_{kt}) - L_s \leq 1 - z_{st}, \quad \forall t \in \mathcal{T}_L \quad (13)$$

Constraint 2c ensures that only one label should be assigned to a particular leaf.

Constraint 2d to 2i are mainly adopted from Bertsimas and Dunn [2019] on Chapter 8.2. Here we briefly explain the meaning and derivations of these constraints. Constraint 2d guarantees that a sample can only be assigned to one leaf node. Constraint 2e and 2f are set to enforce the split of the tree with given training data. They are derived as follow:

First, to enforce the splits required by the structure of the tree when assigning points to leaves, we set constraints:

$$\mathbf{a}_m^T(\mathbf{x}_s + \epsilon) \leq b_m + M_1(1 - z_{st}), \quad \forall m \in A_L(t), t \in \mathcal{T}_L \quad (14)$$

$$\mathbf{a}_m^T \mathbf{x}_s \geq b_m - M_2(1 - z_{st}), \quad \forall m \in A_R(t), t \in \mathcal{T}_L \quad (15)$$

Here ϵ is added to avoid the strict inequality and can be expressed as the smallest non-zero distance between adjacent values of j th feature:

$$\epsilon_j = \min\{x_j^{(i+1)} - x_j^{(i)} \mid x_j^{(i+1)} \neq x_j^{(i)}\} \quad (16)$$

where $x_j^{(i)}$ is the i th largest value in the j th feature.

To specify the M_1 and M_2 , let $\epsilon_{\min} = \min_j\{\epsilon_j\}$ and $\epsilon_{\max} = \max_j\{\epsilon_j\}$, we notice that the largest value for $\mathbf{a}_m^T(\mathbf{x}_s + \epsilon) - b_m$ is $1 + \epsilon_{\max}$. Thus, M_1 can be set to $1 + \epsilon_{\max}$. Similarly, The largest value of $\mathbf{a}_m^T \mathbf{x}_s - b_m$ is 1, so we can set $M_2 = 1$.

Therefore, we reformulate Equation 14 and 15 as follow:

$$\mathbf{a}_m^T(\mathbf{x}_s + \epsilon - \epsilon_{\min}) + \epsilon_{\min} \leq b_m + (1 + \epsilon_{\max})(1 - z_{st}), \quad \forall m \in A_L(t), t \in \mathcal{T}_L \quad (17)$$

$$\mathbf{a}_m^T \mathbf{x}_s \geq b_m - (1 - z_{st}), \quad \forall m \in A_R(t), t \in \mathcal{T}_L \quad (18)$$

Constraint 2g and 2h are set to ensure that if there's no split on $t \in \mathcal{T}_D$, then all samples at this node will go to the right branch of this decision node and send down to the same right most leaf node (along with Constraint 2e and 2f). Constraint 2i enforces the hierarchical structure of the tree. It guarantees a parent node must have a split if its child node has a split.

B Branch-and-Bound Algorithm for Optimal Decision Tree Problem

Algorithm 1 depicts the details of the Branch-and-bound scheme for training the optimal decision tree. Based on Proposition 3 and 4, relaxed MIP and group decomposition provide tighter bounds than the basic lower bound. Therefore, using the basic lower and upper bound is adequate to deduct the convergence proof of Algorithm 1. The critical property of our BB based optimal decision tree training algorithm is that it can guarantee the convergence by only branching on the space of first stage variables $m = (a, b, c, d)$ (i.e. the variables describing the tree structure).

Algorithm 1 can be treated as a rooted BB tree. It starts from a root BB node indexed at level 0 with the original variable space M_0 . Let $\delta(M) = \|m^u - m^l\|_\infty$ be the diameter of the set M . We denote M_{i_q} as the BB node at level q which is explored at iteration i_q . A node $M_{i_{q+1}}$ is a descendent node that connected to its parent node M_{i_q} , with $M_{i_{q+1}} \subset M_{i_q}$. The descendent node is at level $q + 1$ and is explored at iteration i_{q+1} . We denote $\{M_{i_q}\}$ as the sequence of the partition element that represents a path of the BB tree from the root node to the node M_{i_q} at the level q . Since the search space is narrowing along the path, the sequence $\{\beta_i\}$ is monotonically increasing, while $\{\alpha_i\}$ is monotonically decreasing. If $\lim_{i \rightarrow \infty} \alpha_i = \lim_{i \rightarrow \infty} \beta_i = f$, we say the BB algorithm converges. Once the algorithm is convergent, it produces a global ϵ -optimal solution in a finite number of steps.

Algorithm 1: Branch-and-Bound Optimal Decision Tree Algorithm

Input: M_0 , non-zero tolerance ϵ

- 1 Set iteration index $i = 0$, $\mathbb{M} \leftarrow \{M_0\}$;
- 2 Initial upper and lower bounds $\alpha_i = \alpha(M_0)$, $\beta_i = \beta(M_0)$;
- 3 **repeat**
- 4 **Node Selection**
- 5 Select a set $M \in \mathbb{M}$ satisfying $\beta(M) = \beta_i$;
- 6 $\mathbb{M} \leftarrow \mathbb{M} \setminus \{M\}$;
- 7 $i \leftarrow i + 1$;
- 8 **Branching**
- 9 Partition M into subsets M_1 and M_2 according to the branch strategy in Section B.2;
- 10 Add each subset to $\mathbb{M}$ to create separated descendent nodes;
- 11 **Bounding**
- 12 Compute $\alpha(M_1)$, $\beta(M_1)$, $\alpha(M_2)$, $\beta(M_2)$;
- 13 For any $j \in \{1, 2\}$, if $\beta_s(M_j)$ is infeasible for some $s \in \mathcal{S}$, $\mathbb{M} \leftarrow \mathbb{M} \setminus \{M_j\}$;
- 14 $\beta_i \leftarrow \min\{\beta(M') \mid M' \in \mathbb{M}\}$;
- 15 $\alpha_i \leftarrow \min\{\alpha_{i-1}, \alpha(M_1), \alpha(M_2)\}$;
- 16 Remove all M' from $\mathbb{M}$ if $\beta(M') \geq \alpha_i$;
- 17 If $|\beta_i - \alpha_i| \leq \epsilon$, **STOP**;
- 18 **until** $\mathbb{M} = \emptyset$;

B.1 Proof of Theorem 1 (Convergence Analysis)

The proof follows the work of Cao and Zavala [2019] which proves the convergence of the BB algorithm by only branching on first stage variables. In their case, both first stage and second stage variables are continuous variables. They also assume that $Q(\cdot)$ is a continuous function. However, we prove that for optimal decision tree problem, which has mixed-integer first and second stage variables and also discontinuous $Q(\cdot)$ function, such convergence can still hold. We also adopt some basic results from the seminal work in the Chapter IV of Horst and Tuy [2013] and modified definitions and theories accordingly for the optimal decision tree problem.

Specifically, based on Corollary IV.1 and Definition IV.10 from Horst and Tuy [2013], it is easy to see that Algorithm 1 creates *exhaustive subdivisions*. That is, for all decreasing sub-sequences $\{M_{i_q}\}$, $\lim_{q \rightarrow \infty} \delta(M_{i_q}) = 0$. To prove that the lower bound β converges to f , we have to show that the lower bounding operation of Algorithm 1 is *strongly consistent* and its selection operation is *bound improving*.

From Definition IV.6 and Theorem IV.3 of Horst and Tuy [2013], it is trivial that the selection operation in Algorithm 1 is *bound improving*, since it always selects the node with the actual attained

lower bound for further partition. In Lemma 3 of Cao and Zavala [2019], they prove that the basic lower bounding operation is *strongly consistent* if the first stage variables are continuous. In our case, by giving priority to the first stage integer variables, after finite steps, all integer variables are determined and then the prove can be down following Cao and Zavala [2019].

Lemma 5. *Given an exhaustive subdivision on m , Algorithm 1 satisfies $\lim_{i \rightarrow \infty} \beta_i = f$.*

Proof. Since Algorithm 1 uses a *strongly consistent* lower bounding operation and its selection operation is *bound improving*, from Theorem IV.3 in Horst and Tuy [2013], the result is trivial. $\square$

Then, we have to prove the convergence of the upper bounds α through the following lemma.

Lemma 6. *Given an exhaustive subdivision on m , Algorithm 1 generates a sequence $\{\alpha_i\}$ such that $\lim_{i \rightarrow \infty} \alpha_i = f$.*

Proof. Since a, c, d are binary variables and these variables are given priority in the branching strategy, the value of a, c, d will eventually be fixed after finite step of subdivision. Suppose the optimal limit points of them are a^*, c^*, d^* . Therefore, we can have a i_0 , such that $m' = (a^*, b, c^*, d^*)$, $\forall m' \in M_i, \forall i \geq i_0$ with fixed a^*, c^*, d^* . Let $m^* \in M_0$ as an optimal solution of Problem 2, with $m^* = (a^*, b^*, c^*, d^*)$. Without loss of generality, we assume at a decision node t , the j th feature is selected for splitting (i.e. $a_{jt}^* = 1$). Suppose the sorted value of j th feature from small to large is $\{x_{1,j}, \dots, x_{s,j}, \dots, x_{n,j}\}$. Assume $x_{s,j}$ is an optimal solution of b_t . It is easy to see that all values within the range $[x_{s,j}, x_{s+1,j})$ gives the same optimal value. Define b_t^* as the midpoint of the range. Based on the definition of ϵ in Problem 2 (See Chapter 8.2 of Bertsimas and Dunn [2019]), we are guarantee to have the same optimal value on any b such that $b_t \in [b_t^* - 0.5\epsilon_{min}, b_t^* + 0.5\epsilon_{min}]$, $\forall t \in \mathcal{T}_D$. We denote $B(m^*) = \{(a^*, b, c^*, d^*) \mid b_t \in [b_t^* - 0.5\epsilon_{min}, b_t^* + 0.5\epsilon_{min}], \forall t \in \mathcal{T}_D\}$. For every point $m \in B(m^*)$, we have $Q(m) = Q(m^*)$ holds.

Since the subdivision is exhaustive, after a finite number of iterations i' such that $i' \geq i_0$, we have, either the partition considered satisfies $M_{i'} \subseteq B(m^*)$, or the partition $M_{i'}$ which contain the solution m^* is pruned. For the first case, since $M_{i'} \subseteq B(m^*)$, we have $Q(m) = Q(m^*)$, $\forall m \in M_{i'}$. Then we have that $\alpha_{i'} = Q(m^*) = f$. In the second one, since $M_{i'}$ is pruned, then we have $\alpha_{i'} \leq \beta(M_{i'})$. Because $m^* \in M_{i'}$, we also have $\beta(M_{i'}) \leq Q(m^*)$. Hence, $\alpha_{i'} = Q(m^*) = f$. Therefore, $\lim_{i \rightarrow \infty} \alpha_i = f$ holds. $\square$

Combing Lemma 5 and 6, Theorem 1 can be obtained.

B.2 Branching Strategy for Algorithm 1

The branching strategy for Algorithm 1 is designed based on the variables' effect on the structure of the optimal decision tree. Since d indicates whether each decision node is splitting or not, it directly determines the skeleton of the tree structure. Thus, in implementation, the variables of d on each decision node are first used for branching. We devise the following heuristics for the other three decision variables (a, b, c) to select the branching variable: First, we generate a threshold $\tau = 1 - \frac{1}{2}||b^u - b^l||_\infty$. Suppose τ is smaller than a randomly generated number between 0 and 1. In that case, we branch on variable a , which determines the splitting feature of the decision node (If a is all determined, then branching on variable c). When τ is larger than the randomly generated number, we branch on variable b (continuous) at the midpoint of its range. The interior order of each type of variables is based on picking variable with smaller decision(leaf) node index (e.g. $a_{j,t}$ will be chosen over $a_{j,t+1}$, if they both are not determined.)

C Basic Lower Bounding Problem Calculation

Algorithm 2 explicit the way of finding optimal value of the subproblem β_s . It also provides a byproduct $\mathcal{T}_{z_s}$, which indicates the leaf node that sample s can reach under region M . $\mathcal{T}_{z_s}$ can inherit successively to the descendent BB node and directly reduce the number of variables on z for the lower bound calculation. Moreover, $\mathcal{T}_{z_s}$ can also enhance the effect of sample reduction, which further improves the performance of the BB algorithm.

Algorithm 2: Global search for the basic subproblem $\beta_s(M) := \min_{m \in M} Q_s(m)$

Input: $\mathbf{x}_s, \mathbf{y}_s, m^l = (a^l, b^l, c^l, d^l), m^u = (a^u, b^u, c^u, d^u), D, n, \hat{L}, \lambda$

- 1 Set DT root node (decision node) $t_0 = 1$, node set $\mathbb{T} \leftarrow \{t_0\}$, Leaf loss set $\mathbb{L} = \emptyset$, reach leaf node set $\mathcal{T}_{z_s} = \emptyset, \mathcal{T}_D = \{1, \dots, 2^D - 1\}, \mathcal{T}_L = \{2^D, \dots, 2^{D+1} - 1\}$;
- 2 **while** $\mathbb{T} \neq \emptyset$ **do**
- 3 $t = \min\{t \mid t \in \mathbb{T}\}$;
- 4 $\mathbb{T} \leftarrow \mathbb{T} \setminus \{t\}$;
- 5 **if** $t \in \mathcal{T}_D$ **then**
- 6 **if** $d_t^l = d_t^u = 0$ **then**
- 7 **while** $t \notin \mathcal{T}_L$ **do**
- 8 $t \leftarrow 2t + 1$
- 9 **end**
- 10 $\mathbb{T} \leftarrow \mathbb{T} \cup \{t\}$;
- 11 **else**
- 12 $F_t = \{j \mid (a_{jt}^l = a_{jt}^u = 1) \vee (a_{jt}^l \neq a_{jt}^u), \forall j \in \{1, \dots, P\}\}$;
- 13 **for** Each split feature $f \in F_t$ **do**
- 14 **if** $\mathbf{x}_{sf} < b_t^l$ **then**
- 15 $\mathbb{T} \leftarrow \mathbb{T} \cup \{2t\}$;
- 16 **else if** $\mathbf{x}_{sf} \geq b_t^u$ **then**
- 17 $\mathbb{T} \leftarrow \mathbb{T} \cup \{2t + 1\}$;
- 18 **else**
- 19 $\mathbb{T} \leftarrow \mathbb{T} \cup \{2t, 2t + 1\}$; /* $\mathbf{x}_{sf} \in [b_t^l, b_t^u)$ */
- 20 **end**
- 21 **end**
- 22 **end**
- 23 **else**
- 24 $\mathcal{T}_{z_s} \leftarrow \mathcal{T}_{z_s} \cup \{t\}$;
- 25 **if** $y_{sk} \in [c_{kt}^l, c_{kt}^u], \forall k \in \{1, \dots, K\}$ **then**
- 26 $\mathbb{L} \leftarrow \mathbb{L} \cup \{0\}$;
- 27 **else**
- 28 $\mathbb{L} \leftarrow \mathbb{L} \cup \{1\}$;
- 29 **end**
- 30 **end**
- 31 **end**
- 32 **return** $\mathcal{T}_{z_s}, L = \frac{1}{\hat{L}} \min\{l \mid l \in \mathbb{L}\} + \frac{\lambda}{n} \sum_{t \in \mathcal{T}_D} d_t^l$;

C.1 Proof of Theorem 2

In this part we prove Theorem 2 that Algorithm 2 can find all leafs that sample s can fall into under M and can obtain the global optimum for the lower bound subproblem $\beta_s(M)$.

Proof. We proof by induction on maximum depth D :

Base case: Suppose maximum depth $D = 0$, then there's only one leaf node $t = 1$ for the tree model, $\mathcal{T}_z = \{t\}$ under M . The statement is true.

Induction: Suppose the statement is true for the decision tree problem with maximum depth $D \geq 0$ under any specific M , then for decision tree problem with maximum depth $D + 1$, since $D \geq 0$, the first node is a decision node and line 6-22 guarantee to put all its possible descendent nodes into the node set $\mathbb{T}$ according to bound M at decision node t . Since each possible descendent node can be regarded as a root node of decision tree model with depth D , then the decision tree problem rooted with the descendent nodes can find all possible leaf nodes that sample s can fall into with given bound $M = [m^l, m^u]$, according to the assumption. Thus the statement is proved true on decision tree problem with maximum depth $D + 1$.

Therefore, since Algorithm 2 exhaust all possible loss of sample s and pick the minimum one, Algorithm 2 will find the global optimum of subproblem $\beta_s(M) := \min_{m \in M} Q_s(m)$. $\square$

D Additional Analysis for Sample Reduction

In this section, we provide the new formula for two tighter lower bound strategies after sample reduction.

Relaxed MIP With the assistance of $\mathcal{S}_{dt}$ we can further simplify Problem 7 as:

$$\beta^R(M) = \min_{c, L} \sum_{s \in \mathcal{S}_{ud}} \frac{1}{\hat{L}} L_s + \lambda \sum_{t \in \mathcal{T}_D} d_t^l + \sum_{s' \in \mathcal{S}_{dt}} \frac{1}{\hat{L}} L_{s'} \quad (19a)$$

$$\text{s.t. } L_s \geq 1 - \sum_{t \in \mathcal{T}_{z_s}} \sum_{k \in \mathcal{K}} y_{sk} c_{kt} \quad (19b)$$

$$\sum_{k \in \mathcal{K}} c_{kt} = 1, \quad \forall t \in \mathcal{T}_L \quad (19c)$$

$$0 \leq L_s \leq 1 \quad (19d)$$

$$c_{kt} \in \{0, 1\}, \quad \forall t \in \mathcal{T}_L \quad (19e)$$

$$s \in \mathcal{S}_{ud} \quad (19f)$$

Here, $L_{s'}$ are determined according to sample reduction. Compared to Problem 7, the number of variable L_s is reduced from n to $|\mathcal{S}_{ud}|$. The solving efficiency of the second lower bound strategy can be further enhanced.

Group decomposition The computing load of the subproblem formed by each group can also be reduced through sample reduction. Specifically, we can have the following equation for each subproblem β_g^G :

$$\beta_g^G(M) = \min_{m_g \in M, L_s} \sum_{s \in \mathcal{S}_{ud} \cap \mathcal{S}_g} \left(\frac{1}{\hat{L}} L_s + \frac{\lambda}{n} \sum_{t \in \mathcal{T}_D} d_t \right) + \sum_{s' \in \mathcal{S}_{dt} \cap \mathcal{S}_g} \frac{1}{\hat{L}} L_{s'} \quad (20a)$$

$$\text{s.t. Constraint 2g 2h 2i 2j 2k} \quad (20b)$$

$$\frac{1}{2} \sum_{k \in \mathcal{K}} (y_{sk} + c_{kt} - 2y_{sk} c_{kt}) - L_s \leq 1 - z_{st}, \quad \forall t \in \mathcal{T}_{z_s} \quad (20c)$$

$$\sum_{k \in \mathcal{K}} c_{kt} = 1, \quad \forall t \in \mathcal{T}_L \quad (20d)$$

$$\sum_{t \in \mathcal{T}_{z_s}} z_{st} = 1 \quad (20e)$$

$$\mathbf{a}_m^T (\mathbf{x}_s + \epsilon - \epsilon_{min}) + \epsilon_{min} \leq b_m + (1 + \epsilon_{max})(1 - z_{st}), \quad \forall m \in A_L(t), t \in \mathcal{T}_{z_s} \quad (20f)$$

$$\mathbf{a}_m^T \mathbf{x}_s \geq b_m - (1 - z_{st}), \quad \forall m \in A_R(t), t \in \mathcal{T}_{z_s} \quad (20g)$$

$$z_{st} \in \{0, 1\}, \quad \forall t \in \mathcal{T}_{z_s} \quad (20h)$$

$$c_{kt} \in \{0, 1\}, \quad \forall t \in \mathcal{T}_L \quad (20i)$$

$$s \in \mathcal{S}_{ud} \cap \mathcal{S}_g \quad (20j)$$

Here, we can see that the number of decision variables is also vastly reduced. The number of constraints related to samples is significantly reduced accordingly under the scheme of sample reduction. In the case that $|\mathcal{T}_{z_s}| = 1$ for some s , the corresponding constraint 20e- 20g are eliminated on these s , since here z_s is fully determined.

E Additional Experiments

E.1 Results with Hyperparameter Tuning

We implement the experiments with hyperparameter tuning and compared the performance of RS-OCT with CART, OCT [Bertsimas and Dunn, 2017], and FlowOCT [Aghaei et al., 2020]. Since FlowOCT is designed for datasets with binary features, all datasets are transformed to binary with one-hot encoding before feeding to FlowOCT. The code of FlowOCT is implemented in Python[†] and use Gurobi as the MIP solver. Table 3 and 4 present the results of training optimal classification trees on all datasets with hyperparameters selected through validation for all four methods. For OCT and our method, we performed hyperparameter tuning following the instruction in [Bertsimas and Dunn, 2017] to select the continuous regularization parameter λ . For CART, we first train a full decision tree with the maximum depth and then we prune the tree by tuning the pruning-purity parameter from the Julia package `DecisionTree.jl`. For FlowOCT, we followed the instruction in [Aghaei et al., 2020] and modify its regularization parameter λ by selecting values in the set $\{0, 0.1, \dots, 0.9\}$. From the result, we can see that even after hyperparameter tuning, RS-OCT still competitive on most of datasets, compared to other methods. Particularly, for large datasets (i.e., $n \geq 7,000$), when maximum depth of tree is two, RS-OCT can improve the training accuracy of 2.7% and 2.5% on average, compared to CART and OCT, respectively. The testing accuracy (out of sample accuracy) can be improved by 2.8% and 2.5% on average, compared to CART and OCT, respectively.

E.2 Analysis and Ablation Test for Sample Reduction

We provide the analysis and ablation test of the sample reduction on small and median dataset to demonstrate its effectiveness. First, four indicators are presented to help evaluate the effect of sample reduction on each dataset. The NODES represents how many BB nodes are explored during the running of the BB algorithm. The MAX-LEVEL represents the maximum depth of the BB nodes explored by the BB algorithm. The MAX-RATE and AVG-RATE are the indicators of the maximum and average percent of samples whose loss are pre-determined before the calculation of each node. We did not put the MIN-RATE here since, in the root decision node of the tree, no sample can be determined, and thus the MIN-RATE is always zero.

The numerical results (Table 5) reveal that the effect of sample reduction will primarily reduce the computation load when the search tree of the BB algorithm goes deep. Specifically, for datasets (SEEDS, GLASS, BODY, BANKNOTE, WALL-FOLLOWING) that can converge to a small optimality gap (e.g. ≤ 1), the sample reduction devote a significant effort on the calculation process in terms of Max-rate and Avg-Rate. Almost all of these datasets can, on maximum, have the loss determined for over 90% of samples and maintain a relatively high average rate of determined samples among all explored nodes. For the indicator of Nodes and Max-Level, we discover that the exploration of more nodes could also harness the effect of sample reduction even if the node’s level is not deep, especially when the data size is large.

Next, we perform the ablation test for sample reduction. To make the comparison more precise, we only test on datasets where the average percentage of determined samples is above zero. Table 6 presents the test result and reveals that, with sample reduction, all datasets can either converge to a smaller optimality gap within the same time or use less time to reach $<1\%$ optimality gap. Therefore, the sample reduction can have a recognizable effect on improving the calculation efficiency of each BB node.

One limitation of the sample reduction is that it could have reduced influence on datasets with high feature dimensions (e.g. Both two OZONE datasets have 72 features). The reason could be probably due to the large search space of the decision variables (i.e. The number of first-stage variable is $(P + 2)|\mathcal{T}_D| + K|\mathcal{T}_L|$, which is proportional to P and K). Even if the bounds of decision variables is narrowed through the BB procedure, the feasible space under a BB node is still too large to help determine the loss of any samples.

[†]<https://github.com/pashew94/StrongTree>

Table 3: Numerical results on small and median datasets with hyperparameter tuning (Serial).

DATA- SET	n	P	K	METHOD	DEPTH = 2		DEPTH = 3	
					TRAINING	TESTING	TRAINING	TESTING
					ACCURACY (%)	ACCURACY (%)	ACCURACY (%)	ACCURACY (%)
SEEDS	210	7	3	CART	91.2	94.1	91.8	94.1
				OCT	94.3	86.3	96.9	90.2
				FLOWOCT	43.8	30.8	45.3	32.5
				RS-OCT	94.3	88.2	95.0	94.1
GLASS	214	9	6	CART	65.8	64.3	72.2	64.3
				OCT	67.2	64.3	72.8	67.9
				FLOWOCT	55.1	40.7	59.2	37.0
				RS-OCT	68.4	62.5	71.5	62.5
BODY	507	5	2	CART	90.0	88.9	91.9	88.1
				OCT	91.1	87.3	93.7	93.7
				FLOWOCT	60.6	56.7	63.9	59.8
				RS-OCT	92.1	92.9	93.7	91.3
STATLOG- GERMAN	1,000	24	2	CART	73.7	71.2	76.7	74.8
				OCT	73.9	70.0	75.5	72.4
				FLOWOCT	75.7	72.0	77.2	72.0
				RS-OCT	73.9	70.0	76.0	70.8
CONCRETE	1,030	8	3	CART	62.6	61.9	69.5	65.0
				OCT	65.3	63.0	69.9	64.6
				FLOWOCT	57.6	61.2	66.3	65.1
				RS-OCT	65.2	63.8	67.8	70.4
BANKNOTE	1,372	4	2	CART	90.0	89.0	95.1	95.9
				OCT	90.3	87.5	96.2	98.5
				FLOWOCT	54.0	52.2	55.1	52.2
				RS-OCT	92.7	91.3	97.9	96.8
CONTRA- CEPTIVE	1,473	11	3	CART	47.7	46.3	53.4	52.6
				OCT	52.0	52.6	55.8	56.4
				FLOWOCT	50.3	44.4	53.4	46.3
				RS-OCT	52.0	52.6	55.6	56.6
OZONE- EIGHT	1,847	72	2	CART	93.1	93.1	93.7	92.0
				OCT	93.6	93.3	93.7	92.8
				FLOWOCT	93.2	93.7	7.4	6.5
				RS-OCT	93.8	92.6	93.7	92.8
OZONE- ONE	1,848	72	2	CART	97.0	96.8	97.0	96.8
				OCT	97.1	96.3	97.3	96.3
				FLOWOCT	96.8	95.7	96.3	97.2
				RS-OCT	97.3	96.8	97.1	96.1
THYROID- ANN	3,772	21	3	CART	97.8	98.1	99.4	99.0
				OCT	97.8	98.1	99.4	98.8
				FLOWOCT	92.9	94.4	93.0	94.4
				RS-OCT	97.8	98.1	99.4	99.2
STATLOG- LANSAT	4,435	36	6	CART	64.7	64.6	79.6	79.8
				OCT	65.2	64.8	79.6	79.2
				FLOWOCT	31.6	33.0	33.8	33.3
				RS-OCT	66.6	67.8	80.1	79.9
SPAMBASE	4,601	57	2	CART	86.6	86.2	87.9	87.0
				OCT	86.0	84.5	88.6	87.2
				FLOWOCT	81.3	82.8	OOM ¹	OOM
				RS-OCT	85.4	84.5	89.6	89.0
WALL- FOLLOWING	5,456	2	4	CART	94.0	94.0	100.0	100.0
				OCT	94.0	94.0	100.0	100.0
				FLOWOCT	42.5	40.9	43.1	41.4
				RS-OCT	94.0	94.0	100.0	100.0
PAGE- BLOCK	5,473	10	5	CART	95.4	95.0	96.3	95.6
				OCT	95.4	95.3	96.4	95.7
				FLOWOCT	93.9	92.3	OOM	OOM
				RS-OCT	95.4	95.3	96.4	95.7

¹ OUT OF MEMORY.

Table 4: Numerical results on large datasets with hyperparameter tuning (Paralleled with 1000 cores).

DATA-SET	n	P	K	METHOD	DEPTH = 2		DEPTH = 3	
					TRAINING	TESTING	TRAINING	TESTING
					ACCURACY (%)	ACCURACY (%)	ACCURACY (%)	ACCURACY (%)
PENDIGITS	7,494	16	10	CART	38.9	39.1	62.7	62.9
				OCT	39.2	39.4	62.8	63.3
				FlowOCT	36.5	39.0	52.1	53.7
				RS-OCT	39.9	39.9	63.9	63.7
AVILA	10,430	10	12	CART	52.2	52.9	55.0	55.3
				OCT	52.4	53.3	55.0	55.3
				FlowOCT	OOM ¹	OOM	OOM	OOM
				RS-OCT	53.5	54.6	55.0	55.3
EEG	14,980	14	2	CART	62.3	61.7	65.6	63.8
				OCT	62.3	61.7	55.1	55.1
				FlowOCT	55.4	56.3	44.7	43.7
				RS-OCT	66.7	66.0	67.9	65.7
HTRU	17,898	8	2	CART	97.8	97.7	97.8	97.7
				OCT	97.8	97.7	97.8	97.7
				FlowOCT	OOM	OOM	OOM	OOM
				RS-OCT	98.0	97.8	97.9	97.7
SHUTTLE	43,500	9	7	CART	93.9	93.8	99.6	99.6
				OCT	93.9	93.8	99.6	99.6
				FlowOCT	OOM	OOM	OOM	OOM
				RS-OCT	95.4	95.5	99.7	99.8
SKIN-SEGMENTATION	245,057	3	2	CART	90.7	90.6	93.8	93.8
				OCT	90.7	90.6	93.8	93.8
				FlowOCT	OOM	OOM	OOM	OOM
				RS-OCT	92.7	92.7	96.6	96.5
HT-SENSOR	928,991	11	3	CART	58.1	58.1	64.3	64.3
				OCT	58.1	58.1	64.3	64.3
				FlowOCT	OOM	OOM	OOM	OOM
				RS-OCT	59.7	59.8	64.6	64.5

¹ OUT OF MEMORY.

Table 5: The effects of sample reduction on small and median datasets ($D = 2, \lambda = 0.05$, serial).

DATASET	n	P	K	NODES	MAX-LEVEL	MAX-RATE (%)	AVG-RATE (%)
SEEDS	210	7	3	345	38	96.9	35.6
GLASS	214	9	6	178	50	93.7	23.5
BODY	507	5	2	3195	43	100	60.5
STATLOG-GERMAN	1,000	24	2	3736	42	0.0	0.0
CONCRETE	1,030	8	3	22	10	0.0	0.0
BANKNOTE	1,372	4	2	9538	38	99.9	89.9
CONTRACEPTIVE	1,473	11	3	7553	30	84.3	3.9
OZONE-EIGHT	1,847	72	2	2012	73	0.0	0.0
OZONE-ONE	1,848	72	2	6181	83	0.0	0.0
THYROID-ANN	3,772	21	3	23142	79	97.8	0.8
STATLOG-LANSAT	4,435	36	6	1784	45	0.5	0.0
SPAMBASE	4,601	57	2	146	72	0.0	0.0
WALL-FOLLOWING	5,456	2	4	93	38	91.4	19.8
PAGE-BLOCK	5,473	10	5	4227	52	99.0	2.5

Table 6: Ablation test of sample reduction on small and median datasets ($D = 2, \lambda = 0.05$, serial).

DATASET	n	P	K	TIME-No SR	TIME-SR	GAP-No SR	GAP-SR
SEEDS	210	7	3	329.1	151.2	<1%	<1%
GLASS	214	9	6	2,125.6	1,903.8	<1%	<1%
BODY	507	5	2	1,036.1	965.9	<1%	<1%
BANKNOTE	1372	4	2	773.1	633.8	<1%	<1%
WALL-FOLLOWING	5456	2	4	338.3	223.2	<1%	<1%
CONTRACEPTIVE	1473	11	3	14,400.0	14,400.0	12.2%	12.0%
THYROID-ANN	3772	21	3	14,400.0	14,400.0	68.7%	55.6%
PAGE-BLOCK	5473	10	5	14,400.0	14,400.0	47.3%	46.2%